

48. External Accessory Protocol

The External Accessory (EA) framework provides accessories the means to communicate with one or more apps using EA sessions, providing a read/write bytestream interface. Accessory developers specify a custom protocol between the application and the accessory. The design and maintenance of communication protocols between accessories and applications are entirely the responsibility of the accessory and app developers. Apple makes no attempt to secure protocol name space or provide communication security between accessories and applications. External Accessory framework documentation can be found at <https://developer.apple.com/documentation/externalaccessory>.

Accessories may implement this feature by defining and implementing an iAP2 [External Accessory Session](#) (page 491). Additionally, accessories with a [USB Host Mode](#) (page 510) transport component may choose to implement an [EA Native Transport \(USB Host Mode\)](#) (page 353). This generally provides higher performance than the [iAP2 EA Session](#) (page 353). Apple does not guarantee reliable delivery and/or a minimum level of transfer speed.

This feature is supported on tvOS 10.2 or later.

48.1 Requirements

The following requirements apply to accessories supporting this feature:

- Accessories shall declare at least one `SupportedExternalAccessoryProtocol` parameter in their [IdentificationInformation](#) (page 872) message.
- All declared `ExternalAccessoryProtocolIdentifier` and `ExternalAccessoryProtocolName` parameters shall be unique in an [IdentificationInformation](#) (page 872) message.
- Protocol names shall be in reverse-DNS format and shall be associated with domains registered with the Internet Corporation for Assigned Names and Numbers (ICANN), so Apple can rely on each protocol having a unique owner. Apple does not require a developer to be the owner of the domain name used, but does require the developer have permission to use the domain name for the protocol. Protocol names used for demonstration or example purposes in accessory reference designs shall not be used in shipping accessories.
- Accessories shall choose a match action, see [MatchAction](#) (page 874), for each supported protocol. Apple recommends accessories support the [App Match](#) (page 301) feature and use 1 unless the protocol in question is meant for infrequently accessed accessory features, such as firmware updates or diagnostics.

- Accessories shall demonstrate use of all supported protocols during self-certification.

Accessories shall also either implement one iAP2 EA session or one EA Native Transport over a USB Host Mode transport component. Accessories only need to implement one iAP2 EA session regardless of the number of EA protocols supported using the session. Conversely, an EA Native Transport (USB Host Mode) can only support a single EA protocol.

Accessories may implement both an iAP2 EA session for some protocols and use EA Native Transports for others, but an EA protocol shall not use both an iAP2 EA session and an EA Native Transport.

48.1.1 iAP2 EA Session

Accessories supporting the External Accessory Protocol feature using iAP2 shall send or receive the following iAP2 control session message(s):

[StartExternalAccessoryProtocolSession](#) (page 896)

[StopExternalAccessoryProtocolSession](#) (page 896)

To implement an iAP2 EA session, accessories shall also declare the presence of the session during iAP2 link synchronization. Details on the session implementation can be found in [External Accessory Session](#) (page 491).

Accessories supporting the External Accessory Protocol feature using iAP2 may also send or receive the following iAP2 control session message(s):

[StatusExternalAccessoryProtocolSession](#) (page 897)

If the accessory declares support for [StatusExternalAccessoryProtocolSession](#) (page 897), the device will allow multiple iAP2 EA sessions per protocol (excluding EA Native Transport).

48.1.2 EA Native Transport (USB Host Mode)

To implement an EA Native Transport over USB Host Mode, an accessory shall declare one additional USB interface in its device descriptor. The interface shall have two alternate settings; alternate setting 1 shall have one bulk IN and one bulk OUT endpoint, and alternate setting 0 shall be a zero-bandwidth setting with no endpoints. The accessory shall also declare the presence of an EA Native Transport component during identification.

Accessories shall not reuse the same USB Bulk In/Out endpoints for multiple purposes, such as iAP2 and EA native transport. They also shall not shut down the iAP2 connection once a EA Native Transport session has been started.

Table 48-1 USB Interface Descriptor (Alternate Setting 1 - Active Session) for External Accessory Native Transport (USB Host Mode)

USB Descriptor	Value	Comments
Interface Number	0xNN	Shall be different from the iAP2 interface number.
Interface Class	0xFF	Vendor-specific interface
Interface SubClass	0xF0	MFi Accessory
Interface Protocol	0x01	External Accessory Native Transport
Interface String	com.company.protocol	Shall match an identified ExternalAccessoryProtocolName
Number of Endpoints	2	1 Bulk IN and 1 Bulk OUT endpoint descriptor shall be specified
Alternate Setting	1	

Table 48-2 USB Interface Descriptor (Alternate Setting 0 - Zero Bandwidth) for External Accessory Native Transport (USB Host Mode)

USB Descriptor	Value	Comments
Interface Number	0xNN	Shall be the same as the default interface
Interface Class	0xFF	Vendor-specific interface
Interface SubClass	0xF0	MFi Accessory
Interface Protocol	0x01	External Accessory Native Transport
Interface String	com.company.protocol	Shall match an identified ExternalAccessoryProtocolName
Number of Endpoints	0	
Alternate Setting	0	

48.2 Usage

48.2.1 iAP2 EA Session Usage

iAP2 EA Session protocol initiation is always driven by an application running on the device. When the accessory receives a [StartExternalAccessoryProtocolSession](#) (page 896) message, it may start to send and receive traffic on the identified iAP2 EA Session for the specified protocol and session identifier.

Conversely, the accessory shall cease sending and receiving traffic for the specified session identifier when a [StopExternalAccessoryProtocolSession](#) (page 896) message is received. Device-powered accessories may also be required to transition to a low power state, see [Accessory Power \(Lightning\)](#) (page 295).

48.2.2 EA Native Transport (USB Host Mode) Usage

If it is using EA Native Transport over USB Host Mode, the accessory's default interface will be selected by the device when an application opens an EA session with the accessory. Once selected, the accessory shall continually service both the bulk IN and OUT endpoints to maintain communication with the app. When the application has closed the EA session, the alternate zero-bandwidth interface will be selected to signal the accessory when EA traffic for the protocol has ceased. Device-powered accessories may also be required to transition to a low power state, see [Accessory Power \(Lightning\)](#) (page 295).

EA Native Transport (USB Host Mode) performance will vary based on many factors including, but not limited to:

- Full Speed USB versus High Speed USB.
- Bulk endpoint packet size.
- Depth of USB packet queues.
- Other concurrent USB traffic on the same bus/controller.

48.2.3 EA Session Management

If more than one iOS application will support an accessory's EA protocol, the accessory shall support the possibility of multiple iOS applications attempting to open an EA session to the accessory at the same time.

Accessories shall support the [StatusExternalAccessoryProtocolSession](#) (page 897) message to manage EA sessions if an accessory needs to:

- Enforce a limit on the maximum number of iOS applications communicating with the accessory at any given time.
- Close an open EA session immediately due to external factors such as impending accessory shutdown.

48.2.4 iOS Application Usage

iOS applications shall use the External Accessory framework to communicate with EA accessories. Complete documentation for the iOS External Accessory framework can be found at <https://developer.apple.com/documentation/externalaccessory>.

48.2.5 Firmware Updates

Apple recommends supporting accessory firmware updates from a companion iOS application. Firmware updates from an application enables field issues to be resolved without requiring return of the accessory to a service center. New accessory features and capabilities can also be deployed in the field using the application.

48.2.6 Background Operation

Backgrounded iOS applications may continue to communicate with EA accessories using the External Accessory framework, see https://developer.apple.com/documentation/uikit/app_and_environment/scenes/preparing_your_ui_to_run_in_the_background/about_the_background_execution_sequence.

48.2.7 Notification Center

iOS applications communicating with accessories using the External Accessory framework should consider notifying the user of accessory status changes using the iOS Notification Center. This may improve user awareness of accessory state while using other iOS applications.

Additionally, companion iOS applications should consider enabling users to quickly interact with and control accessories using one or more Notification Center widget app extensions.

Information on use of the Notification Center framework can be found at <https://developer.apple.com/documentation/notificationcenter>.

48.2.8 File Provider Extensions

Accessories viewable as a filesystem should consider implementing support for a File Provider extension using a companion iOS application and the External Accessory Protocol.

Implementing a File Provider extension enables compatible iOS applications to view or edit files stored on the accessory.

Additional information on File Provider extensions can be found at <https://developer.apple.com/documentation/fileprovider>.

48.2.9 Shortcuts Extensions

Companion iOS applications should consider donating shortcuts for accessory actions using Intents and IntentsUI app extensions.

Donating shortcuts enables users to:

- Interact with the accessory from the Siri interface and other applications such as Shortcuts and Maps.
- Include accessory actions in custom-defined shortcuts.

Additional information on shortcuts extensions can be found at:

- https://developer.apple.com/documentation/sirikit/donating_shortcuts
- https://developer.apple.com/documentation/sirikit/deleting_donated_shortcuts
- https://developer.apple.com/documentation/sirikit/soup_chef_accelerating_app_interactions_with_shortcuts

- https://developer.apple.com/documentation/sirikit/shortcut-related_ui
- https://developer.apple.com/documentation/sirikit/watch_and_widget_support
- <https://developer.apple.com/documentation/sirikit/media>

48.2.10 Bluetooth Pairing

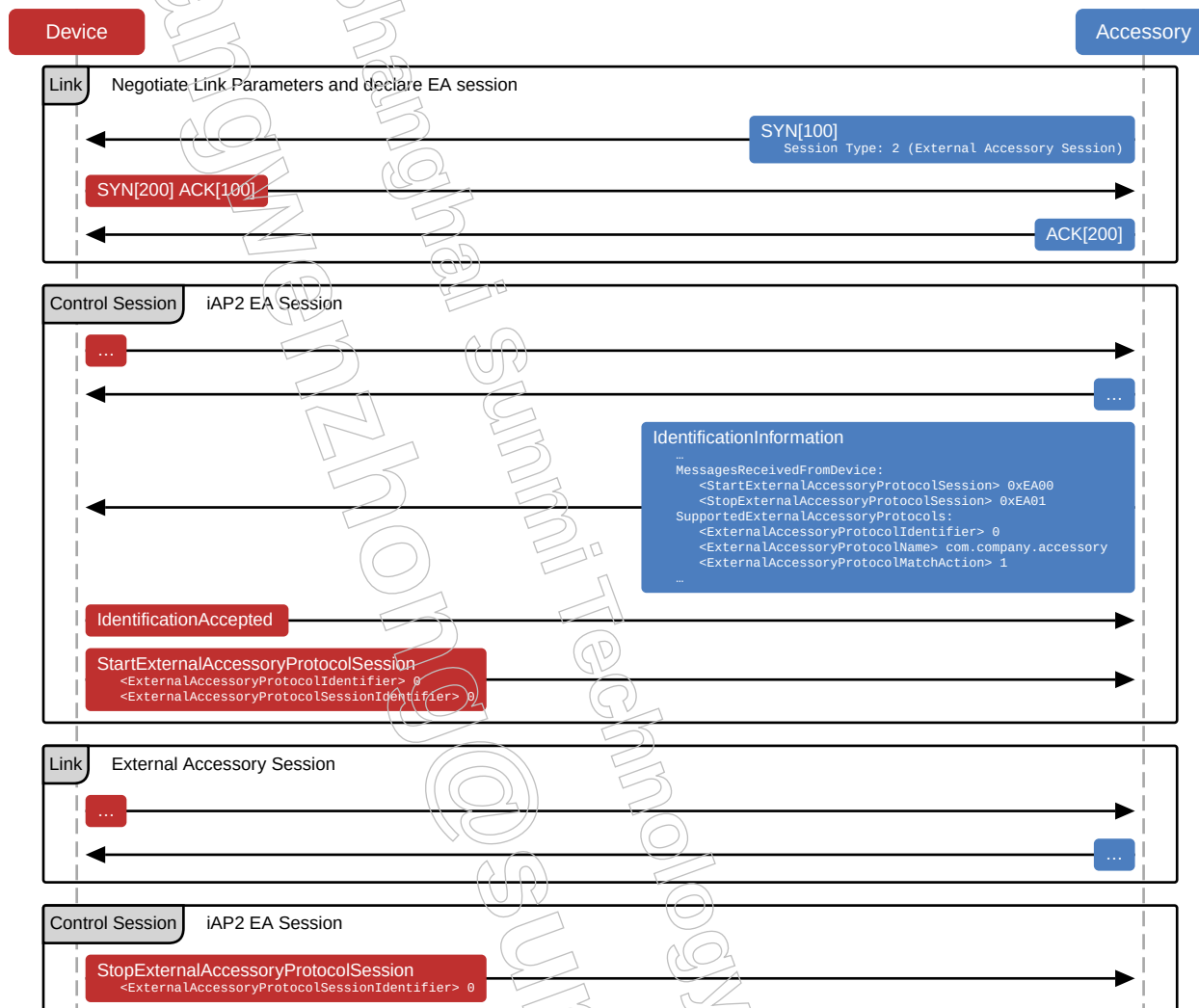
If the accessory implements iAP2 over Bluetooth and has a companion iOS application, but does not support automated Bluetooth pairing using [Out-of-Band Bluetooth Pairing](#) (page 416) using another iAP2 transport, the iOS application should not direct the user to manually pair with the accessory using *Settings > Bluetooth*.

Instead, the application should use the External Accessory framework to scan for and initiate Bluetooth pairing with the accessory, see:

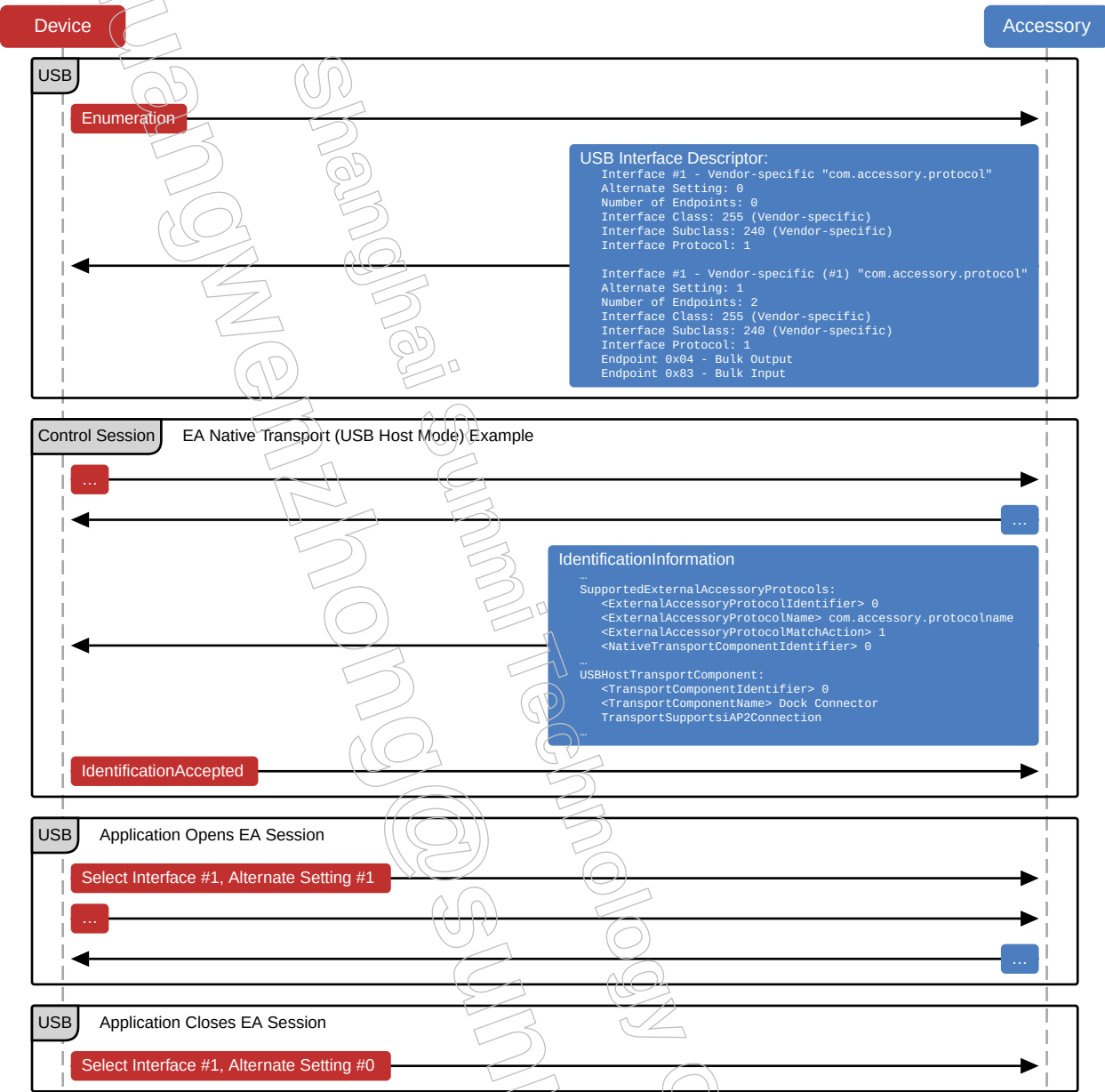
- [iAP2](#) (page 536)
- [https://developer.apple.com/documentation/externalaccessory/eaaccessorymanager/showbluetoothaccessorypicker\(withnamefilter:completion:\)](https://developer.apple.com/documentation/externalaccessory/eaaccessorymanager/showbluetoothaccessorypicker(withnamefilter:completion:))

48.3 Examples

48.3.1 iAP2 EA Session Example

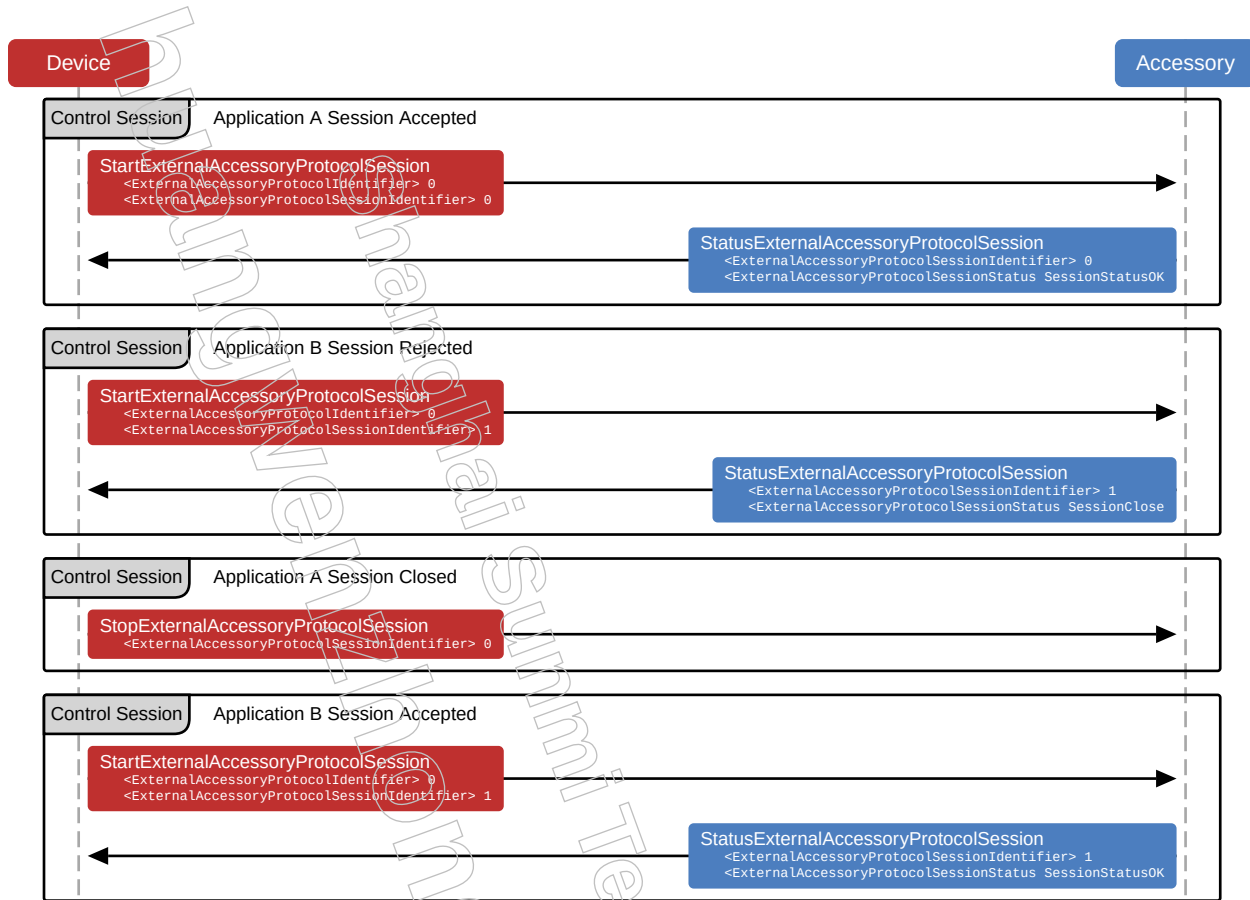


48.3.2 EA Native Transport (USB Host Mode) Example



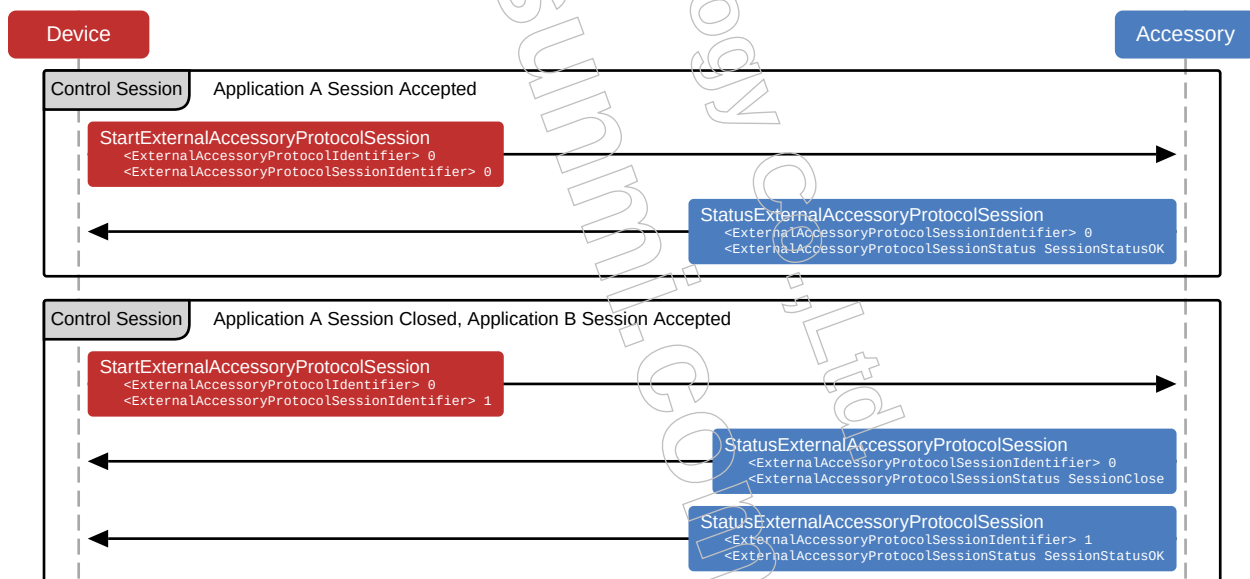
48.3.3 EA Session Management (First In Wins) Example

This example illustrates how an accessory can enforce a 'first in wins' session management policy for an EA protocol limited to one active session at a time.



48.3.4 EA Session Management (Last In Wins) Example

This example illustrates how an accessory can enforce a 'last in wins' session management policy for an EA protocol limited to one active session at a time.



48.4 Test Procedures

48.4.1 SDK Apps

The tests below are for accessories communicating with iOS apps.

48.4.1.1 Equipment

- Latest released version of Xcode.
- Mac running OS X 10.10 Yosemite or later.
- Latest revision of ATS software.
- ATS hardware.
- BPA100 hardware for Bluetooth capture.
- iOS application to be tested.
- iOS application in either .ipa or .app format.

48.4.1.2 Procedure

1. Verify there are no errors in ATS when opening and closing app.
2. Connect a device not supporting the External Accessory protocol, such as iPod nano. Verify there are no errors in ATS.
3. From within ATS, note the value for ExternalAccessoryProtocolName inside IdentificationInformation. The value should be in the form of *com.name.protocol*.
4. On the Mac, control-click the .app file and select "Show Package Contents".
5. Open "info.plist" with Xcode and note the value for "Supported External Accessory Protocols".
6. Verify the "Supported External Accessory Protocols" value matches the value from within ATS.

48.4.2 External Accessory Protocol over iAP2

1. Verify the accessory declares the presence of an External Accessory session in the Link Packets traffic view.
2. Verify RequestAppLaunch is only sent in response to a direct user action; this does not apply to automotive head units.

48.4.3 External Accessory Protocol over Native

1. Verify StartExternalAccessoryProtocol and StopExternalAccessoryProtocol are not listed as declared messages in IdentificationInformation (unless the accessory also uses External Accessory Protocol over iAP2).
2. Verify there is at least one SupportedExternalAccessoryProtocol group IdentificationInformation.

3. Verify there is at least one NativeTransportComponentIdentifier in the IdentificationInformation message SupportedExternalAccessoryProtocol group.
4. Verify each NativeTransportComponentIdentifier corresponds to a declared USBHostTransportComponent group (and the TransportComponentIdentifier matches).
5. Verify the External Accessory Protocol interface name in the USB descriptor (in the USB Transfers traffic view) matches the protocol name sent in IdentificationInformation.